Project Setup & Architecture

Table of Contents

---

1. Local Setup

```
# Prerequisites: Node 20+, Expo CLI, EAS CLI
npm install -g expo-cli eas-cli

# Clone and install dependencies
git clone https://github.com/bsecure/user-app.git
cd bsecure-user-app
npm install

# iOS native dependencies (macOS only)
npx pod-install ios

# Copy environment file
cp .env.example .env

# Start Metro bundler
npx expo start

# Run on specific platforms
npx expo run:ios          # iOS simulator
npx expo run:android      # Android emulator
npx expo start --tunnel   # Physical device via Expo Go
```

---

2. Environment Variables

Environment variables are injected at build time via `expo-constants` using the `extra` field in `app.json`. Never commit `.env` to version control.

`.env` file:

```
API_BASE_URL=https://api.bsecure.in/api/v1
WS_BASE_URL=wss://api.bsecure.in
GOOGLE_MAPS_API_KEY=AIzaSy...
RAZORPAY_KEY_ID=rzp_live_...
SENTRY_DSN=https://abc123@o123.ingest.sentry.io/456
```

`app.config.ts` (dynamic config to read `.env`):

```ts
import 'dotenv/config';
import { ExpoConfig, ConfigContext } from 'expo/config';

export default ({ config }: ConfigContext): ExpoConfig => ({
  ...config,
  name: 'b-secure',
  slug: 'bsecure-user',
  extra: {
    apiBaseUrl: process.env.API_BASE_URL,
    wsBaseUrl: process.env.WS_BASE_URL,
    googleMapsApiKey: process.env.GOOGLE_MAPS_API_KEY,
    razorpayKeyId: process.env.RAZORPAY_KEY_ID,
    sentryDsn: process.env.SENTRY_DSN,
    eas: { projectId: 'your-eas-project-id' },
  },
});
```

`src/lib/constants.ts` — typed access to env vars:

```ts
import Constants from 'expo-constants';

const extra = Constants.expoConfig?.extra as {
  apiBaseUrl: string;
  wsBaseUrl: string;
  googleMapsApiKey: string;
  razorpayKeyId: string;
  sentryDsn: string;
};

export const ENV = {
  API_BASE_URL: extra.apiBaseUrl,
  WS_BASE_URL: extra.wsBaseUrl,
  GOOGLE_MAPS_API_KEY: extra.googleMapsApiKey,
  RAZORPAY_KEY_ID: extra.razorpayKeyId,
  SENTRY_DSN: extra.sentryDsn,
} as const;
```

---

3. app.json Configuration

```
{
  "expo": {
    "name": "b-secure",
    "slug": "bsecure-user",
    "version": "1.0.0",
    "scheme": "bsecure",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "splash": {
      "image": "./assets/images/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#0f172a"
    },
    "ios": {
      "bundleIdentifier": "in.bsecure.user",
      "supportsTablet": false,
      "infoPlist": {
        "NSLocationWhenInUseUsageDescription": "b-secure needs
        your location to find nearby guards.",
        "NSLocationAlwaysUsageDescription": "b-secure needs your
        location to track active bookings.",
        "NSCameraUsageDescription": "b-secure needs camera access
        for profile photo.",
        "NSPhotoLibraryUsageDescription": "b-secure needs photo
        library access for profile photo."
      }
    },
    "android": {
      "package": "in.bsecure.user",
      "adaptiveIcon": {
        "foregroundImage": "./assets/images/adaptive-icon.png",
        "backgroundColor": "#0f172a"
      },
      "permissions": [
        "ACCESS_FINE_LOCATION",
        "ACCESS_COARSE_LOCATION",
        "ACCESS_BACKGROUND_LOCATION",
        "CAMERA",
        "READ_EXTERNAL_STORAGE",
        "WRITE_EXTERNAL_STORAGE",
        "VIBRATE"
      ]
    },
    "plugins": [
      [
        "expo-location",
        {
          "locationAlwaysAndWhenInUsePermission": "b-secure needs
        location access to show nearby guards and track active
        bookings.",
          "isIosBackgroundLocationEnabled": true
```

```
        }
      ],
      [
        "expo-notifications",
        {
          "icon": "./assets/images/notification-icon.png",
          "color": "#0f172a",
          "sounds": ["./assets/sounds/notification.wav"]
        }
      ],
      [
        "expo-camera",
        {
          "cameraPermission": "b-secure needs camera access for
         your profile photo."
        }
      ],
      "expo-secure-store",
      "expo-font",
      "@sentry/react-native/expo"
    ],
    "experiments": {
      "typedRoutes": true
    }
  }
}
```

---

4. Expo Router Layout

Root Layout — `app/_layout.tsx`

```
import { useEffect } from 'react';
import { Stack } from 'expo-router';
import { QueryClientProvider } from '@tanstack/react-query';
import { GestureHandlerRootView } from 'react-native-gesture-
handler';
import { BottomSheetModalProvider } from '@gorhom/bottom-sheet';
import * as SplashScreen from 'expo-splash-screen';
import * as Sentry from '@sentry/react-native';
import { queryClient } from '@/lib/queryClient';
import { useAuthStore } from '@/store/useAuthStore';
import { usePushNotifications } from '@/hooks/
usePushNotifications';
import { ENV } from '@/lib/constants';

SplashScreen.preventAutoHideAsync();

Sentry.init({
  dsn: ENV.SENTRY_DSN,
  tracesSampleRate: 0.2,
```

```
});

export default function RootLayout() {
  const { token, hydrate, hydrated } = useAuthStore();
  usePushNotifications();

  useEffect(() => {
    hydrate(); // Loads JWT from SecureStore on app start
  }, []);

  useEffect(() => {
    if (hydrated) SplashScreen.hideAsync();
  }, [hydrated]);

  if (!hydrated) return null;

  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <QueryClientProvider client={queryClient}>
        <BottomSheetModalProvider>
          <Stack screenOptions={{ headerShown: false }}>
            {token ? (
              // Authenticated: show tabs
              <Stack.Screen name="(tabs)" />
            ) : (
              // Unauthenticated: show auth screens
              <Stack.Screen name="(auth)" />
            )}
            <Stack.Screen name="guards/[id]"
options={{ presentation: 'modal' }} />
            <Stack.Screen name="booking/create"
options={{ presentation: 'modal' }} />
            <Stack.Screen name="booking/tracking" />
            <Stack.Screen name="booking/[id]" />
            <Stack.Screen name="sos" options={{ presentation:
'fullScreenModal' }} />
          </Stack>
        </BottomSheetModalProvider>
      </QueryClientProvider>
    </GestureHandlerRootView>
  );
}
```

Auth Stack Layout — `app/(auth)/_layout.tsx`

```
import { Stack } from 'expo-router';

export default function AuthLayout() {
  return (
    <Stack
      screenOptions={{
        headerShown: false,
        animation: 'slide_from_right',
```

```
      }}
    />
  );
}
```

Tabs Layout — `app/(tabs)/_layout.tsx`

```
import { Tabs } from 'expo-router';
import { Home, Calendar, Wallet, User } from 'lucide-react-
native';
import { useBookingStore } from '@/store/useBookingStore';
import { View, Text } from 'react-native';

export default function TabsLayout() {
  const { activeBooking } = useBookingStore();

  return (
    <Tabs
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: '#0f172a',
        tabBarInactiveTintColor: '#94a3b8',
        tabBarStyle: {
          height: 64,
          paddingBottom: 8,
          paddingTop: 8,
        },
      }}
    >
      <Tabs.Screen
        name="home"
        options={{
          title: 'Home',
          tabBarIcon: ({ color, size }) => <Home color={color}
size={size} />,
        }}
      />
      <Tabs.Screen
        name="bookings"
        options={{
          title: 'Bookings',
          tabBarIcon: ({ color, size }) => <Calendar color={color}
size={size} />,
          tabBarBadge: activeBooking ? '•' : undefined,
        }}
      />
      <Tabs.Screen
        name="wallet"
        options={{
          title: 'Wallet',
          tabBarIcon: ({ color, size }) => <Wallet color={color}
size={size} />,
        }}
```

```
      />
      <Tabs.Screen
        name="profile"
        options={{
          title: 'Profile',
          tabBarIcon: ({ color, size }) => <User color={color}
size={size} />,
        }}
      />
    </Tabs>
  );
}
```

---

5. Global State (Zustand)

src/store/useAuthStore.ts

```
import { create } from 'zustand';
import * as SecureStore from 'expo-secure-store';

const TOKEN_KEY = 'bsecure_jwt';
const REFRESH_KEY = 'bsecure_refresh';

export interface AuthUser {
  id: number;
  name: string;
  phone: string;
  email: string | null;
  profilePhoto: string | null;
}

interface AuthState {
  token: string | null;
  refreshToken: string | null;
  user: AuthUser | null;
  hydrated: boolean;

  hydrate: () => Promise<void>;
  login: (token: string, refreshToken: string, user: AuthUser) =>
        Promise<void>;
  logout: () => Promise<void>;
  setUser: (user: AuthUser) => void;
  setToken: (token: string) => Promise<void>;
}

export const useAuthStore = create<AuthState>((set) => ({
  token: null,
  refreshToken: null,
  user: null,
  hydrated: false,
```

```
  hydrate: async () => {
    try {
      const [token, refreshToken] = await Promise.all([
        SecureStore.getItemAsync(TOKEN_KEY),
        SecureStore.getItemAsync(REFRESH_KEY),
      ]);
      set({ token, refreshToken, hydrated: true });
    } catch {
      set({ hydrated: true });
    }
  },

  login: async (token, refreshToken, user) => {
    await Promise.all([
      SecureStore.setItemAsync(TOKEN_KEY, token),
      SecureStore.setItemAsync(REFRESH_KEY, refreshToken),
    ]);
    set({ token, refreshToken, user });
  },

  logout: async () => {
    await Promise.all([
      SecureStore.deleteItemAsync(TOKEN_KEY),
      SecureStore.deleteItemAsync(REFRESH_KEY),
    ]);
    set({ token: null, refreshToken: null, user: null });
  },

  setUser: (user) => set({ user }),

  setToken: async (token) => {
    await SecureStore.setItemAsync(TOKEN_KEY, token);
    set({ token });
  },
}));
```

src/store/useBookingStore.ts

```
import { create } from 'zustand';
import { Booking } from '@/types';

interface BookingState {
  activeBooking: Booking | null;
  setActiveBooking: (booking: Booking | null) => void;
  updateActiveBookingStatus: (status: Booking['status']) => void;
}

export const useBookingStore = create<BookingState>((set) => ({
  activeBooking: null,

  setActiveBooking: (booking) => set({ activeBooking: booking }),
```

```
  updateActiveBookingStatus: (status) =>
    set((state) => ({
      activeBooking: state.activeBooking
        ? { ...state.activeBooking, status }
        : null,
    })),
}));
```

src/store/useLocationStore.ts

```
import { create } from 'zustand';

export interface LocationCoords {
  latitude: number;
  longitude: number;
  accuracy: number | null;
  heading: number | null;
}

interface LocationState {
  currentLocation: LocationCoords | null;
  setCurrentLocation: (location: LocationCoords) => void;
}

export const useLocationStore = create<LocationState>((set) => ({
  currentLocation: null,
  setCurrentLocation: (location) => set({ currentLocation:
        location }),
}));
```

---

6. Axios Instance

src/api/axios.ts

```
import axios, {
  AxiosInstance,
  AxiosError,
  InternalAxiosRequestConfig,
  AxiosResponse,
} from 'axios';
import * as SecureStore from 'expo-secure-store';
import { router } from 'expo-router';
import { ENV } from '@/lib/constants';

const TOKEN_KEY = 'bsecure_jwt';
const REFRESH_KEY = 'bsecure_refresh';

// Prevent concurrent refresh calls
```

```ts
let isRefreshing = false;
let failedQueue: Array<{
  resolve: (token: string) => void;
  reject: (err: unknown) => void;
}> = [];

const processQueue = (error: unknown, token: string | null) => {
  failedQueue.forEach(({ resolve, reject }) => {
    if (token) resolve(token);
    else reject(error);
  });
  failedQueue = [];
};

export const apiClient: AxiosInstance = axios.create({
  baseURL: ENV.API_BASE_URL,
  timeout: 15_000,
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json',
  },
});

// ── Request interceptor: attach JWT ──────────────────────────
apiClient.interceptors.request.use(
  async (config: InternalAxiosRequestConfig) => {
    const token = await SecureStore.getItemAsync(TOKEN_KEY);
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error),
);

// ── Response interceptor: handle 401, token refresh ──────────
apiClient.interceptors.response.use(
  (response: AxiosResponse) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as
        InternalAxiosRequestConfig & {
      _retry?: boolean;
    };

    if (error.response?.status === 401 && !
        originalRequest._retry) {
      if (isRefreshing) {
        // Queue request until refresh completes
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject });
        }).then((token) => {
```

```
          if (originalRequest.headers) {
            originalRequest.headers.Authorization = `Bearer $
        {token}`;
          }
          return apiClient(originalRequest);
        });
      }

      originalRequest._retry = true;
      isRefreshing = true;

      try {
        const refreshToken = await
        SecureStore.getItemAsync(REFRESH_KEY);
        if (!refreshToken) throw new Error('No refresh token');

        const { data } = await axios.post<{ access: string }>(
          `${ENV.API_BASE_URL}/auth/token/refresh/`,
          { refresh: refreshToken },
        );

        await SecureStore.setItemAsync(TOKEN_KEY, data.access);
        processQueue(null, data.access);

        if (originalRequest.headers) {
          originalRequest.headers.Authorization = `Bearer $
        {data.access}`;
        }
        return apiClient(originalRequest);
      } catch (refreshError) {
        processQueue(refreshError, null);
        // Clear tokens and redirect to login
        await Promise.all([
          SecureStore.deleteItemAsync(TOKEN_KEY),
          SecureStore.deleteItemAsync(REFRESH_KEY),
        ]);
        router.replace('/(auth)/login');
        return Promise.reject(refreshError);
      } finally {
        isRefreshing = false;
      }
    }

    return Promise.reject(error);
  },
);

export default apiClient;
```

7. React Query Setup

src/lib/queryClient.ts

```ts
import { QueryClient } from '@tanstack/react-query';

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 2,      // 2 minutes — data is fresh
      gcTime: 1000 * 60 * 10,         // 10 minutes — cache
        garbage collection
      retry: (failureCount, error: any) => {
        // Do not retry on 4xx client errors
        if (error?.response?.status >= 400 &&
        error?.response?.status < 500) {
          return false;
        }
        return failureCount < 2;
      },
      retryDelay: (attemptIndex) => Math.min(1000 * 2 **
        attemptIndex, 10_000),
      refetchOnWindowFocus: false,
      refetchOnReconnect: true,
    },
    mutations: {
      retry: 0,
    },
  },
});
```

---

8. TypeScript Interfaces

src/types/index.ts

```ts
// ── Enums ──────────────────────────────────────────────

export enum BookingType {
  ON_DEMAND = 'on_demand',
  SCHEDULED = 'scheduled',
}

export enum BookingStatus {
  PENDING = 'pending',
  GUARD_ASSIGNED = 'guard_assigned',
  GUARD_EN_ROUTE = 'guard_en_route',
  GUARD_ARRIVED = 'guard_arrived',
```

```
  ACTIVE = 'active',
  COMPLETED = 'completed',
  CANCELLED = 'cancelled',
}

export enum GuardTier {
  BASIC = 'basic',
  STANDARD = 'standard',
  PREMIUM = 'premium',
  ELITE = 'elite',
}

export enum TransactionType {
  CREDIT = 'credit',
  DEBIT = 'debit',
}

export enum TransactionStatus {
  PENDING = 'pending',
  SUCCESS = 'success',
  FAILED = 'failed',
  REFUNDED = 'refunded',
}

// ── Core Models ──────────────────────────────────────────────

export interface User {
  id: number;
  name: string;
  phone: string;
  email: string | null;
  profilePhoto: string | null;
  createdAt: string;
}

export interface GuardLocation {
  latitude: number;
  longitude: number;
  heading: number | null;
  updatedAt: string;
}

export interface Guard {
  id: number;
  name: string;
  phone: string;
  profilePhoto: string | null;
  tier: GuardTier;
  rating: number;
  totalReviews: number;
  skills: string[];
  bio: string;
```

```ts
  isAvailable: boolean;
  distance?:
        number;           // metres, present in nearby-guards list
  location?: GuardLocation;
}

export interface BookingAddress {
  address: string;
  latitude: number;
  longitude: number;
  landmark?: string;
}

export interface BookingTimelineEvent {
  status: BookingStatus;
  timestamp: string;
  note?: string;
}

export interface Booking {
  id: number;
  user: Pick<User, 'id' | 'name' | 'phone'>;
  guard: Guard | null;
  type: BookingType;
  status: BookingStatus;
  pickupAddress: BookingAddress;
  scheduledAt: string | null;
  startedAt: string | null;
  endedAt: string | null;
  durationMinutes: number | null;
  ratePerHour: number;
  platformFee: number;
  totalAmount: number;
  paidFromWallet: number;
  notes: string;
  timeline: BookingTimelineEvent[];
  createdAt: string;
}

export interface Transaction {
  id: number;
  type: TransactionType;
  status: TransactionStatus;
  amount: number;
  description: string;
  razorpayOrderId: string | null;
  razorpayPaymentId: string | null;
  bookingId: number | null;
  createdAt: string;
}

export interface Wallet {
  id: number;
```

```ts
  userId: number;
  balance: number;
  currency: string;
  updatedAt: string;
}

export interface Review {
  id: number;
  bookingId: number;
  userId: number;
  userName: string;
  userPhoto: string | null;
  rating: number;
  comment: string;
  createdAt: string;
}

// ── API Request/Response Shapes ──────────────────────────

export interface PaginatedResponse<T> {
  count: number;
  next: string | null;
  previous: string | null;
  results: T[];
}

export interface OTPRequestPayload {
  phone: string;
  countryCode: string;
}

export interface OTPVerifyPayload {
  phone: string;
  countryCode: string;
  otp: string;
}

export interface AuthResponse {
  access: string;
  refresh: string;
  user: User;
}

export interface CreateBookingPayload {
  guardId: number;
  type: BookingType;
  pickupAddress: BookingAddress;
  scheduledAt?: string;
  notes?: string;
}

export interface TopUpPayload {
```

```
  amount: number;
}

export interface TopUpResponse {
  orderId: string;
  amount: number;
  currency: string;
  keyId: string;
}

export interface ConfirmTopUpPayload {
  orderId: string;
  paymentId: string;
  signature: string;
}

export interface SubmitReviewPayload {
  bookingId: number;
  rating: number;
  comment: string;
}

export interface SOSTriggerPayload {
  bookingId: number;
  latitude: number;
  longitude: number;
}
```

---

9. ESLint, Prettier & Path Aliases

tsconfig.json

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    }
  },
  "include": ["**/*.ts", "**/*.tsx", ".expo/types/**/*.d.ts",
        "expo-env.d.ts"]
}
```

.eslintrc.js

```
module.exports = {
  extends: [
```

```
    'expo',
    'prettier',
    'plugin:@tanstack/eslint-plugin-query/recommended',
  ],
  plugins: ['prettier'],
  rules: {
    'prettier/prettier': 'error',
    'no-console': ['warn', { allow: ['warn', 'error'] }],
    '@typescript-eslint/no-explicit-any': 'warn',
    '@typescript-eslint/consistent-type-imports': 'error',
  },
};
```

.prettierrc

```
{
  "semi": true,
  "singleQuote": true,
  "trailingComma": "all",
  "printWidth": 90,
  "tabWidth": 2,
  "bracketSpacing": true
}
```

babel.config.js

```
module.exports = function (api) {
  api.cache(true);
  return {
    presets: ['babel-preset-expo'],
    plugins: [
      'nativewind/babel',
      [
        'module-resolver',
        {
          root: ['./src'],
          alias: { '@': './src' },
        },
      ],
      'react-native-reanimated/plugin', // must be last
    ],
  };
};
```